



CS4051NI/CC4059NI Fundamentals of Computing

70% Individual Coursework

2024/25 Spring

Student Name: Poonam Maharjan

London Met ID: 24046427

College ID: NP01NT4A240140

Assignment Due Date: Wednesday, May 14, 2025

Assignment Submission Date: Wednesday, May 14, 2025

Word Count: 3550

Project File Links:

Onedrive Drive Link:	Keep Google Drive URL of your Project Here with Anyone in Organization can View Option Enabled
-----------------------------	--

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

24046427 Poonam Maharjan2.docx

 Islington College,Nepal

Document Details

Submission ID

trn:oid:::3618:95781706

Submission Date

May 14, 2025, 9:53 AM GMT+5:45

Download Date

May 14, 2025, 9:54 AM GMT+5:45

File Name

24046427 Poonam Maharjan2.docx

File Size

22.9 KB

31 Pages

3,547 Words

18,366 Characters

20% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **27 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
-  **18 Missing Quotations 12%**
Matches that are still very similar to source material
-  **1 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 13%  Internet sources
- 0%  Publications
- 15%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 27 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
- 18 Missing Quotations 12%**
Matches that are still very similar to source material
- 1 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 13% Internet sources
- 0% Publications
- 15% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
www.geeksforgeeks.org		4%
2	Internet	
fotc.com		3%

Table of Contents

1. Introduction	1
1.1 Introduction:.....	1
1.2 Aim.....	2
1.3 Objectives	2
1.4 Technologies/Tools Used	2
2. Discussion and Analysis.....	7
2.1 Algorithm.....	7
2.2 Flowchart.....	8
2.3 Pseudocode:	10
2.3.1 For read.py.....	10
2.3.2 For write.py.....	10
2.3.3 For operation.py	11
2.3.4 For main.py	13
2.4 Data Structures	14
2.4.1 Primitive Data Type:.....	14
2.4.2 Non-Primitive Data Type:.....	15
3. Program	18
4. Testing	26
4.1 Test 1	26
4.2 Test 2	27
4.3 Test 3	29
4.4 Test 4	31
4.5 Test 5	34
5. Conclusion.....	38
6. Appendix.....	39
7. Bibliography	45

List of Figures

Figure 1 Technologies/Tool	2
Figure 2 Python tools used	3
Figure 3 Notepad and text in notepad file	3
Figure 4 IDLE (Python 3.13) (unstop, 2025).	4
Figure 5 Draw.io	5
Figure 6 Microsoft word	5
Figure 7 Flow Chart (Wambua, 2023).	8
Figure 8 Flowchart of the Program	9
Figure 9 Primitive & Non-Primitive Data type	14
Figure 10 10 Inside poroducts.txt	18
Figure 11 read.py in shell	18
Figure 12 Write.py in shell	19
Figure 13 1st for selling products in operate.py	20
Figure 14 2nd selling products in opration.py	21
Figure 15 1st restock the products in shell	22
Figure 16 2nd restock the products in shell	22
Figure 17 generation invoice after sale	23
Figure 18 generating invoice after restocking	23
Figure 19 Main.py in shell	24
Figure 20 Exiting the program	25
Figure 21 try and except for validation	26
Figure 22 Negative value for sale	27
Figure 23 Invalid ID for sale	27
Figure 24 Negative value for purchase	28
Figure 25 Invalid ID for purchase	28
Figure 26 Invoice generation for purchasing in shell	30
Figure 27 Purchased multiple items saved in .txt file	31
Figure 28 Invoice generation for sales	32
Figure 29 Sold products saved in .txt file	33
Figure 30 Invoice creation for first sell product	35
Figure 31 Product updated on next sell	35
Figure 32 Products shown in .txt file	36
Figure 33 Product added and updated in shell	36
Figure 34 34 Updated shown in .txt file	37

List of Tables

<i>Table 1 List description</i>	16
<i>Table 2 Dictionary description</i>	16
<i>Table 3 Sets description</i>	17
<i>Table 4 Tuples description</i>	17
<i>Table 5 Test 1</i>	26
<i>Table 6 test 2</i>	27
<i>Table 7 test 3</i>	29
<i>Table 8 test 4</i>	31
<i>Table 9 test 5</i>	34

1. Introduction

1.1 Introduction:

This coursework is about developing a Skin Care product, to a local vendor that sells beauty and skin care products, where there should be the system for products sales, restocking and more. This coursework is done by a python programming language, which reads, writes, the product.txt file and manages the overall deals, updates the stocks and generates an invoice for both sales and restocking. It enhances and brings out a real-life scenario, like promotional offers Buy 3 Get 1 free, and handling the sell sales and restocking of the products.

This assignment uses python programming to build, a notepad to store the products, IDLE shell to write the python program, Draw.io to draw flowchart of the program, and Microsoft word to write the report. The key features which should be in the program is that it should sell the products using the logics, should be able to generate invoices for both sell and restock uniquely, and should also update the stock for the products. The feature for the user, using it should be the options of the products should be displayed until the user exits the program. This report also needs to add algorithms, flow charts and pseudocode, as it is used to describe the high-level language code in simple, visualized form.

This course taught about, data structures like List and Dictionary, loops, and solving a real-world scenario. It also taught to read and write to external .txt file, which is used to log the transactions. All the algorithms, flowcharts and pseudo helped to clear and learn the plan. Mostly, it helped to how to make an user friendly application, handle the exceptions, to apply logics, to create different invoices (bills), totally indicating real-world scenario.

1.2 Aim

The aim of the project is to build a functioning program a sales management system for WeCare which is for beauty and skincare shop, by using python programming language.

1.3 Objectives

The objective of this project is to fulfil:

- Read the beauty and skincare product details from the file and store it.
- Providing the products to the users.
- Managing the sales of the products, and using buy three and get one free, policy and update stock after buying each time.
- Providing the invoices, to the customers and suppliers for the products.
- Restocking the products and updating the prices and quantities.

1.4 Technologies/Tools Used

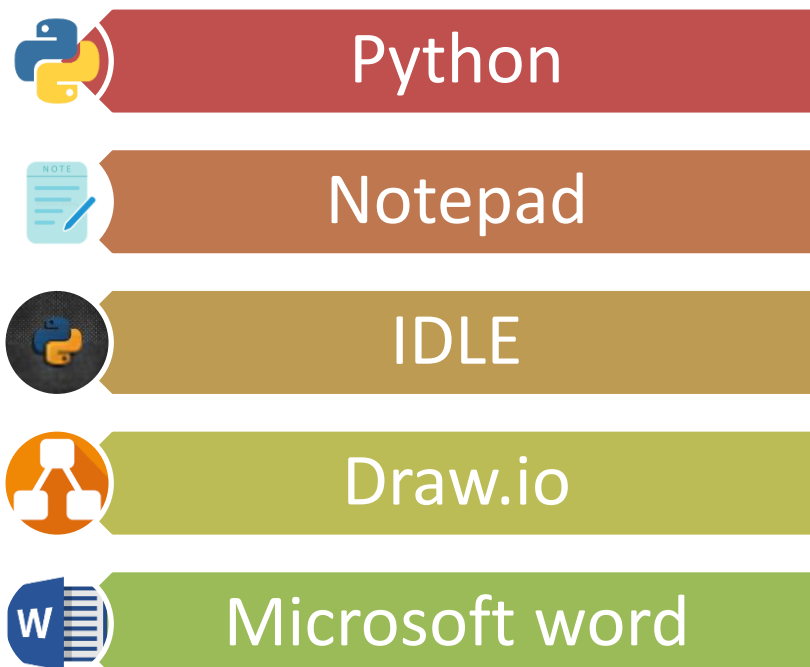


Figure 1 Technologies/Tool

Python: Python is an interpreted, object-oriented, high-level programming language (python, 2025). In this coursework, python 3.13 version has been used. In simple words, Python is a computer programming language often used to build websites and software's, automate tasks, and analyse data. Python is a general-purpose language, used to create various programmes and isn't specialised for any specific problems (Coursera , 2023).



Figure 2 Python tools used

Due to its similarity with English syntax, python is widely considered to be one of the easiest programming languages to write, read, and learn. Another reason python is more popular.

Notepad: Notepad is a simple text editor, which allows you to create, edit, and store text files (geeksforgeeks, 2024). The file after saved is stored in .txt extension. In our coursework we have used one .txt file, to store products, where there is the product name, company name, quantity, price and country.

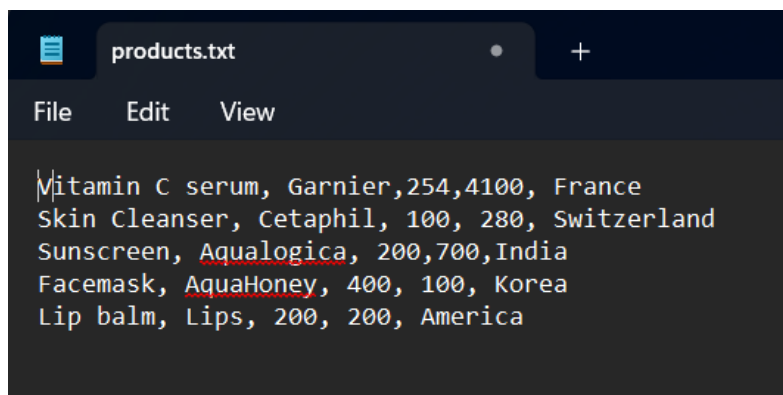


Figure 3 Notepad and text in notepad file

In simple words, Notepad is designed to be lightweight and a simple text editor. (lenovo, 2025). It provides essential and most needed features like creating, opening, and saving the text file. There are also basic editing functionalities, such as copy, cut, paste, undo and redo. It supports searching for specific words or phrase within a document (lenovo, 2025).

IDLE: IDLE allows, cross-platform, python shell window with colourful code input, output and error messages , it provides multi-window text editor with multiple undo, smart indent, and other extra features (docs.python.org, 2025).

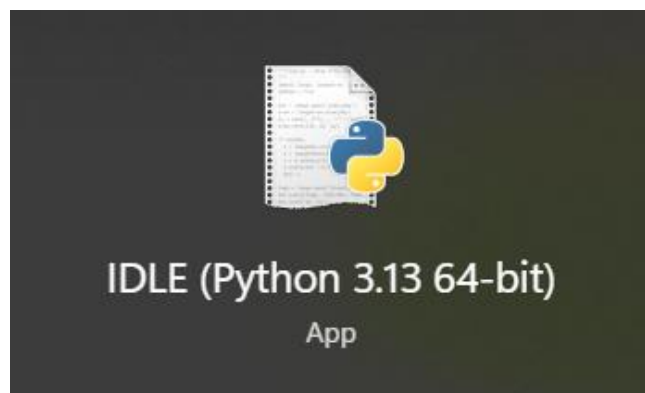


Figure 4 IDLE (Python 3.13) (unstop, 2025).

Python IDLE is a development environment that comes bundled with the language itself. This IDE packs a punch and lets the programmers to write, run, debug, and execute the code with ease (unstop, 2025). It serves as a convenient tool for writing, testing and debugging python code (unstop, 2025). It is extremely user-friendly, due to features like intelligent indentation and highlights (unstop, 2025) .

Draw.io: Draw.io is a free, online diagramming tool that allows you to create flowcharts, diagrams, and mind maps, organisation charts, and much more (Paraschiv, 2023). This is an open source, self-hosted, diagramming tool that can help your team create visual charts using pre-designed templates on a drag-and-drop interface, while also being able to import and export diagrams in various formats (Paraschiv, 2023).



Figure 5 Draw.io

This application is used to make flow charts for our coursework. It helps in illustrating the process of the project. Draw.io is also used for mapping out ideas or concepts via a mind map, visualizing a network using a network diagram, sketching out the designs of a website or user interface, creating a floor plan or layout diagram as proof of concept (Paraschiv, 2023).

Microsoft word: MS word is a word processor developed by Microsoft, as it has advance features which allow you to format and edit your files and documents in the best possible way (byjus, 2025). The extension for any word file .doc or .docx, it is used to make professional-quality documents, reports (byjus, 2025).



Figure 6 Microsoft word

For the coursework, Microsoft word is used to make reports. The key feature of Microsoft word is, it is very easy to use and has easy to use interface, it provides real time checking of grammar and spelling, pling, provides formatting with different fonts and colours, there is the option for many smart tables and many more.

2. Discussion and Analysis

2.1 Algorithm

it is a process used for solving problem and performing a compilation. algorithm acts as an exact list of instructions that conduct specified action step by step (Gillis, 2025).

Step 1: Start

Step 2: Reads the details from product.txt

Step 3: Store the details in a dictionary

Step 4: Display welcome messages

Step 5: Display the options to the customers

Step 6: Enter 1 to sell products

Step 7: Take user input

Step 8: Ask for username and phone

Step 9: Ask for a valid productid, and quantity

Step 10: Apply discounts

Step 11: Calculate the stocks

Step 12: Create and save the invoice after purchase

Step 13: Go to step 4

Step 14: Enter 2 to restock products

Step 15: Ask for the username and phone

Step 16: Display the available products, quantities and price

Step 17: Ask for valid productid and quantity

Step 18: Ask for new cost prices if necessary

Step 19: Create and save the invoice after restocking

Step 20: Go to step 4

Step 21: Display a text message and terminate the program

Step 22: End

2.2 Flowchart

A flowchart is a diagram where it uses various shapes to represent the use and design of the overall blueprint of the program.

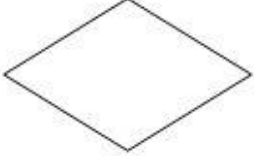
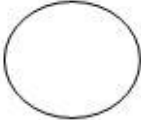
	Terminal/terminator	Indicates start/end of the flowchart or process
	Decision	Represents different decisions emerging from different points
	Action/Process	Represents an action or process
	Input/output	Holds the input/output information
	Connector	Indicates the flow connection to the next symbol

Figure 7 Flow Chart (Wambua, 2023).

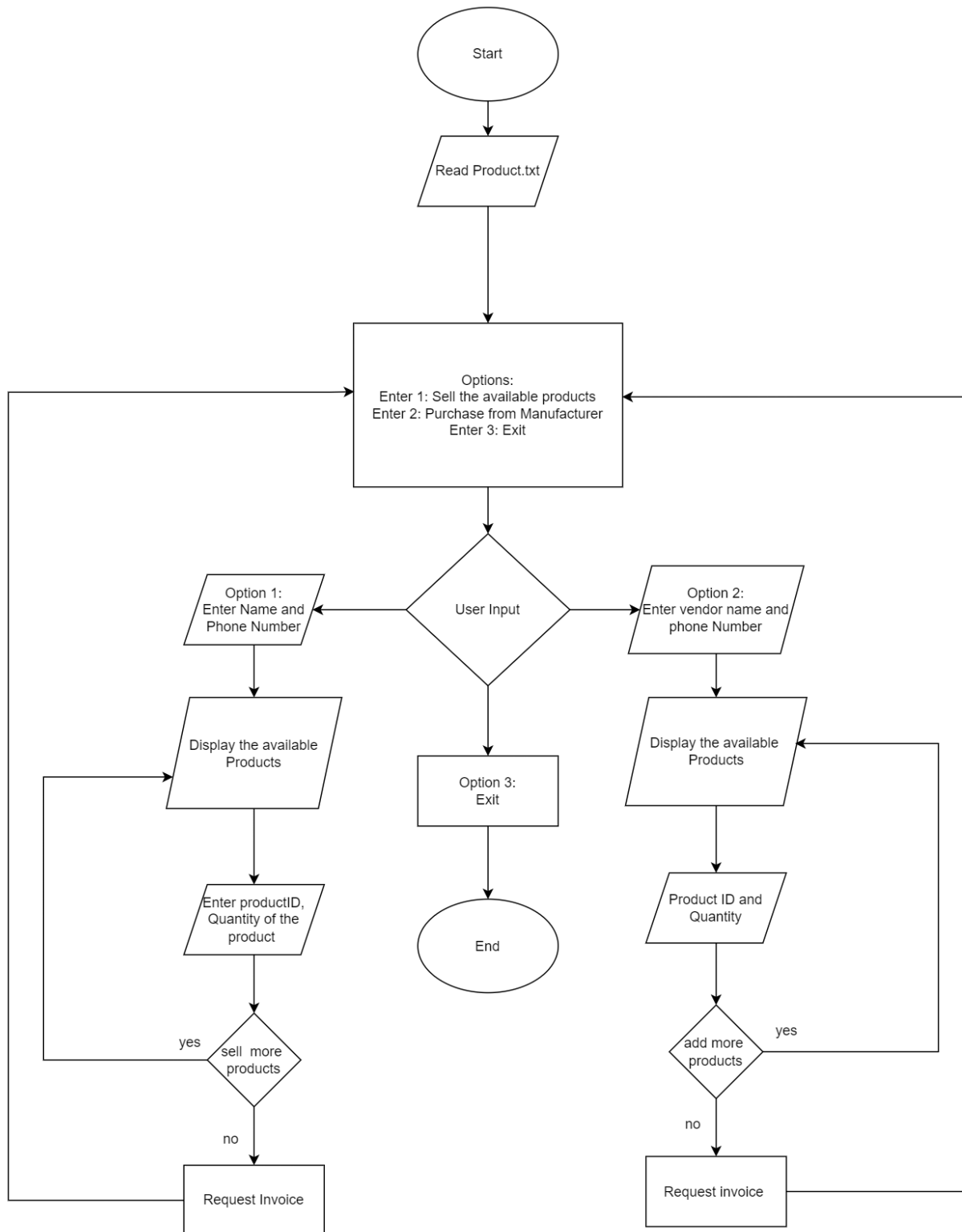


Figure 8 Flowchart of the Program

2.3 Pseudocode:

2.3.1 For read.py

```

DEFINE FUNCTION read_products with filename products.txt
    INITIALIZE an empty dictionary named products
    TRY
        OPEN filename and READ all line into info
        SET product_name = 1
        FOR each line in info"
            REMOVE newline, SPLIT by comma
            INCREMENT product_number
    EXCEPT FileNotFoundError:
        PRINT "File not found"
    RETURN products
END FUNCTION

```

2.3.2 For write.py

```

DEFINE FUNCTION write_products with parameter products, and file product.txt
    START FUNCTION
        TRY
            OPEN filename in write mode as file
            FOR each product_info in products:
                JOIN product_info with comma into lines
                WRITE line to file with newline
            EXCEPT exception as e:
                PRINT "Error writing to file"
        END TRY
    END FUNCTION

```

Invoice_generation

```

DEFINE FUNCTION invoice_generation with parameters (transaction_type,
customer_name, phone, items, total)
START FUNCTION
    TRY
        GET current date and time
        CREATE unique filename using transaction_type and timestamp
        OPEN filename in write mode as file
        WRITE invoice title and transaction time to file
        WRITE customer name and phone
        FOR each item in items:
            FORMAT item details Into line
            WRITE line to file
        WRITE total amount
    
```

```

        PRINT invoice filename
    EXCEPT any error:
        PRINT "Error generating invoice"
    END TRY
END FUNCTION

```

2.3.3 For operation.py

```

IMPORT invoice_generation, write_products from Write
DEFINE FUNCTION sell_product with parameter products
TRY
    LOOP to check valid customer_name:
        INPUT name
        CHECKS name
    LOOP until valid phone_number:
        INPUT phone number
        CHECKS phone
INITIALIZE cart empty, grand_total 0
    LOOP
        DISPLAY all available Products
    LOOP until valid selected_id
        INPUT product ID
        CHECKS id and exists in product.txt
    TRY
        INPUT quantity
    EXCEPT ValueError:
        PRINT "Invalid quantity. Enter integer"
    CALCULATE free_items = quantity // 3
    CALCULATE total_required = quantity + free_items

    IF total_required > available_stock:
        PRINT "not sufficient Quantity"
        CONTINUE
    UPDATE stock = stock - total_required
    CALCULATE unit_price = price * 3
    CALCULATE item_total = unit_price * quantity
    ADD grand_total to item_total

    ASK if user wants to sell more
    IF not 'y':
        BREAK
    CALL invoice_generation
EXCEPT exception as e:
    PRINT "An error occurred"
END FUNCTION

```

Restocking**DEFINE FUNCTION** restock_products**START FUNCTION****TRY****LOOP** to check valid vendor_name:**INPUT** vendor_name**CHECK** name**LOOP** to check valid phone**INPUT** phone**CHECK** number**INITIALIZE** cart empty, total_cost 0**LOOP****DISPLAY** available products from product.txt**LOOP** to check valid selected_id:**INPUT** product ID**CHECK** id**TRY****INPUT** quantity**EXCEPT** ValueError:**PRINT** "Invalid quantity. Please enter a number/ digits."**CONTINUE****INPUT** new price (optional)**IF** price entered:**UPDATE** price of product**CALCULATE** total = unit_price * quantity**ADD** total to total_cost**UPDATE** product**APPEND** product details to cart**ASK** if users want to restock more**IF** not y**BREAK****CALL** invoice_generation**EXCEPT** Exception as e:**PRINT** "Error during restocking"**END FUNCTION**

2.3.4 For main.py

```
IMPORT read_products from read
IMPORT write_products from write
IMPORT sell_product, restock_product from operation

DEFINE FUNCTION main
START FUNCTION
    DISPLAY "welcome to WeCare Wholesale System"
    CALL read_products

    LOOP
        DISPLAY menu options
        TRY
            GET user input as choice
            IF choice == 1:
                CALL sell_product(products)
            ELIF choice == 2:
                CALL restock_product(products)
            ELIF choice == 3:
                CALL write_products(products)
                PRINT "Thank you and visit again"
                BREAK
        ELSE:
            PRINT "Invalid Option, Please Choose Valid option"
        EXCEPT ValueError:
            PRINT "Please choose valid Option"

END FUNCTION
```

2.4 Data Structures

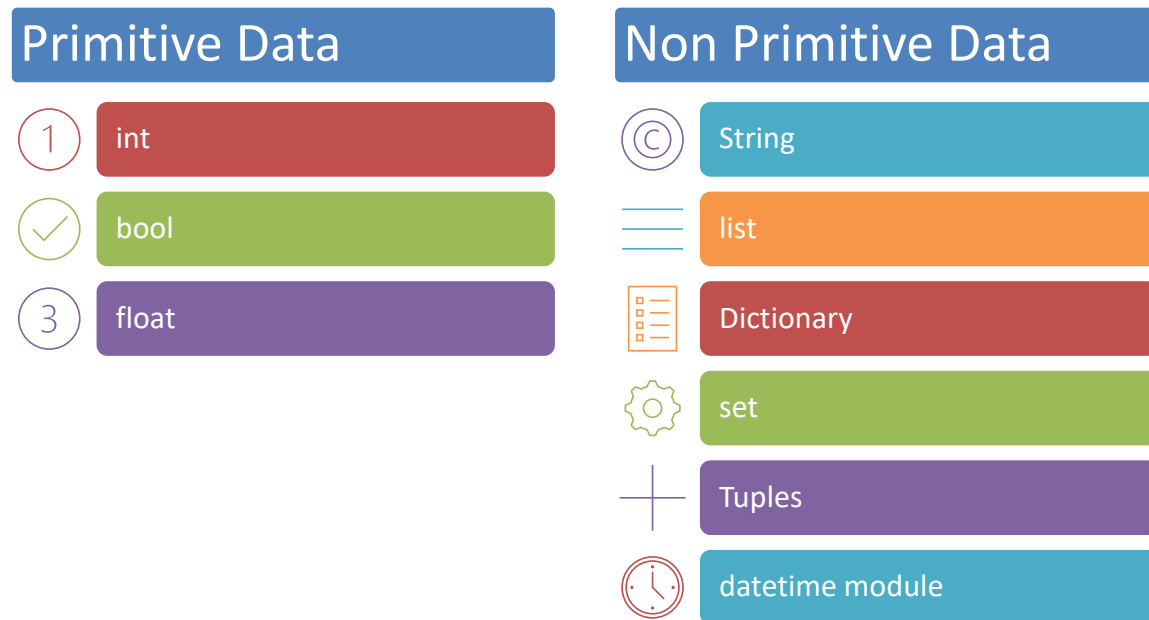


Figure 9 Primitive & Non-Primitive Data type

2.4.1 Primitive Data Type: Primitive Data type is predefined, it can have certain values, and its size depends on the type of data structure, for example: int, double, float and more (geeksforgeeks, 2024). It can be easily identified as primitive data types are in lowercase.

Int: Integer Data Type (int), denotes that the given variable can store the integer numbers (geeksforgeeks, 2025). This keyword used to import numeric values, both positive and negative. Its size is 4-bytes so, it's for 64-bit system. It can store value from -2,147,483,648 to 2,147,483,647 (geeksforgeeks, 2025).

Float: A data type which is used to store floating-point numbers with single precision, it occupies 4 bytes which is 32 bits of memory and provides approximately 7 digits of precision, it is an integer data type but with decimals, it might provide errors while dealing with large numbers (geeksforgeeks, 2025). It is less costly in terms of memory uses, and it also requires less memory space as compared to double data type (geeksforgeeks,

2025). Float can also be scientific number with an “e” to indicate the power of 10 (w3schools, 2025).

Boolean (bool): A Boolean is a data type with two possible values: true (1) or false (0), the two values help represent truth conditions found in logic control structures (Dancuk, 2023). Python also has many built-in functions that return to Boolean value, like `isinstance()` function, which can be used to determine if an object is of a certain data type (w3schools, 2025).

2.4.2 Non-Primitive Data Type: Non-Primitive Data Types are not predefined. It is created by users or programmers, Non-Primitive Data types can be null (geeksforgeeks, 2024). Its size is not fixed as primitive which depends on the data type. To identify a non-primitive data type, it starts with the uppercase.

String (str): In python, strings are used for representing textual data, it is a sequence of characters enclosed in either single quotes (") or a double quote (") (ElHousieny, 2023). The python language provides various built-in methods and functionalities to work with string efficiently; programmers are also able to concatenate or combine string using the '+' operator (ElHousieny, 2023).

List: A list is a built-in dynamic size array which grows and shrinks, we can store all types of item (including another list) in a list, a list may contain mixed type of items, this is because a list can stores all type of data's (geeksforgeeks, 2025). A list can contain duplicate items, lists are mutable as we can modify and replace or delete the items, list are also ordered, it also maintain the order of elements based on how they are added, items are accessed through the indexing which starts from 0 (geeksforgeeks, 2025). List are created with big brackets [].

Terms	Description
Mutable	Programmers can delete, modify, and replace the items
Ordered	The items have a defined order, and that order will not change, if the programmer adds new items on the list, the item will be set at the end.
Duplicate	A list can contain all types of data, for example string, integer, char and more.

Table 1 List description

Dictionary: A python dictionary is a data structure that stores the value in key: value pairs, Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be immutable (geeksforgeeks, 2025). It is created with curly bracket { }.

Terms	Description
Key	A key must be unique within the dictionary, if one assigns the same value as the last assigned one will overwrite the previous value.
Values	A value can be unique, and of any data type.
Duplicate	Dictionary can contain all data types with keys and values, for example: A = {1: 'Cream', 2: 'Lip balm'}
mutable	Programmers can add, modify or change the value they created.

Table 2 Dictionary description

Sets: A set-in python is used to store a collection of items. No duplicate elements are allowed to add, it is unordered collection, and sets are mutable (geeksforgeeks, 2025). Sets are created by curly bracket { }.

Terms	Descriptions
Duplicate	No duplicate data types are allowed in python; it is only unique elements
Unordered	Set is unordered, it means the elements are accessed without specifying the index like in lists or with keys in dictionary.
Mutable	Elements can be added and removed after creating a set.

Table 3 Sets description

Tuples: A tuple in python is an immutable, ordered collection of elements, it is very similar to lists, but tuples can't be changed after their creations, they cannot be extended or appended (geeksforgeeks, 2025). Tuples are created with small bracket ().

Terms	Descriptions
Immutable	It means once the tuple is created, it's element can't be changed, added or removed.
Ordered	The items or elements ordered will not change its position.

Table 4 Tuples description

3. Program

The program starts with a file structure and in a sequence, read.py it reads the product data from the file. The function read_product is made, and it opens the products.txt file and loads the product information into a dictionary, each product consists of product name, brand, quantity, price and country.

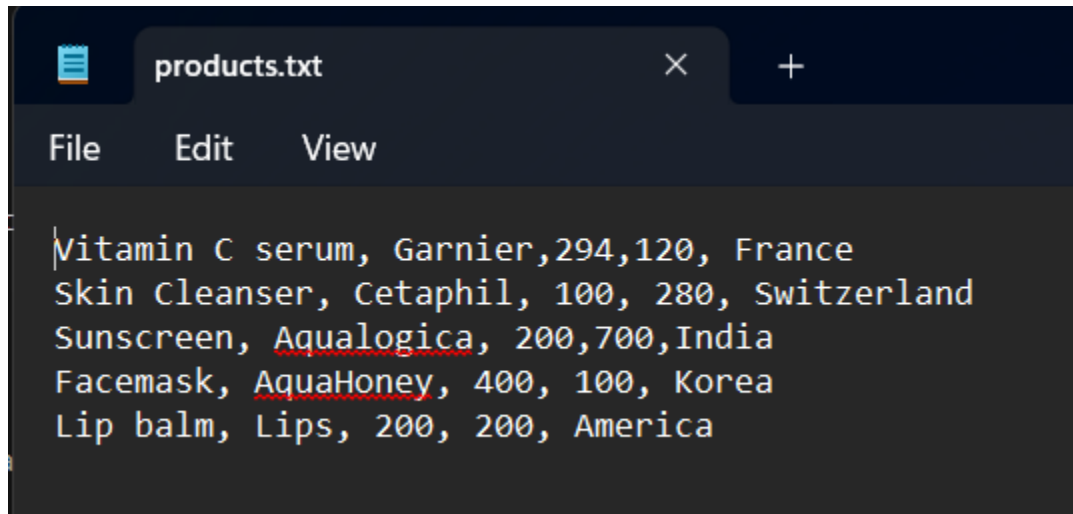


Figure 10 10 Inside poroducts.txt

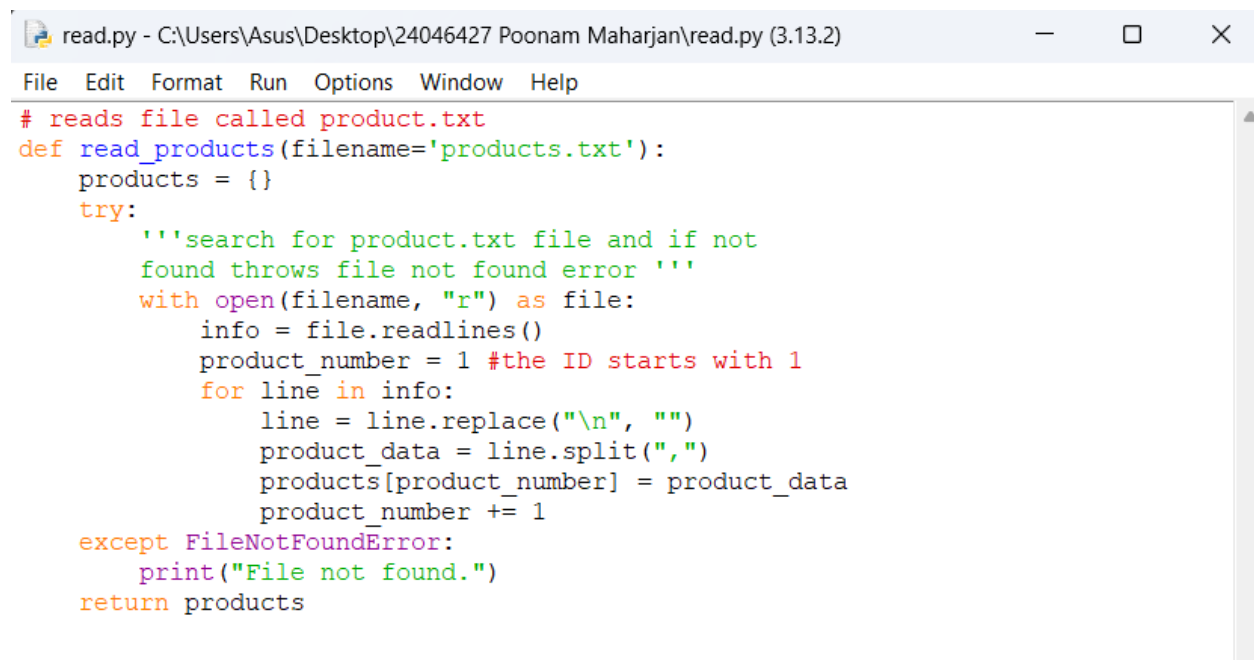
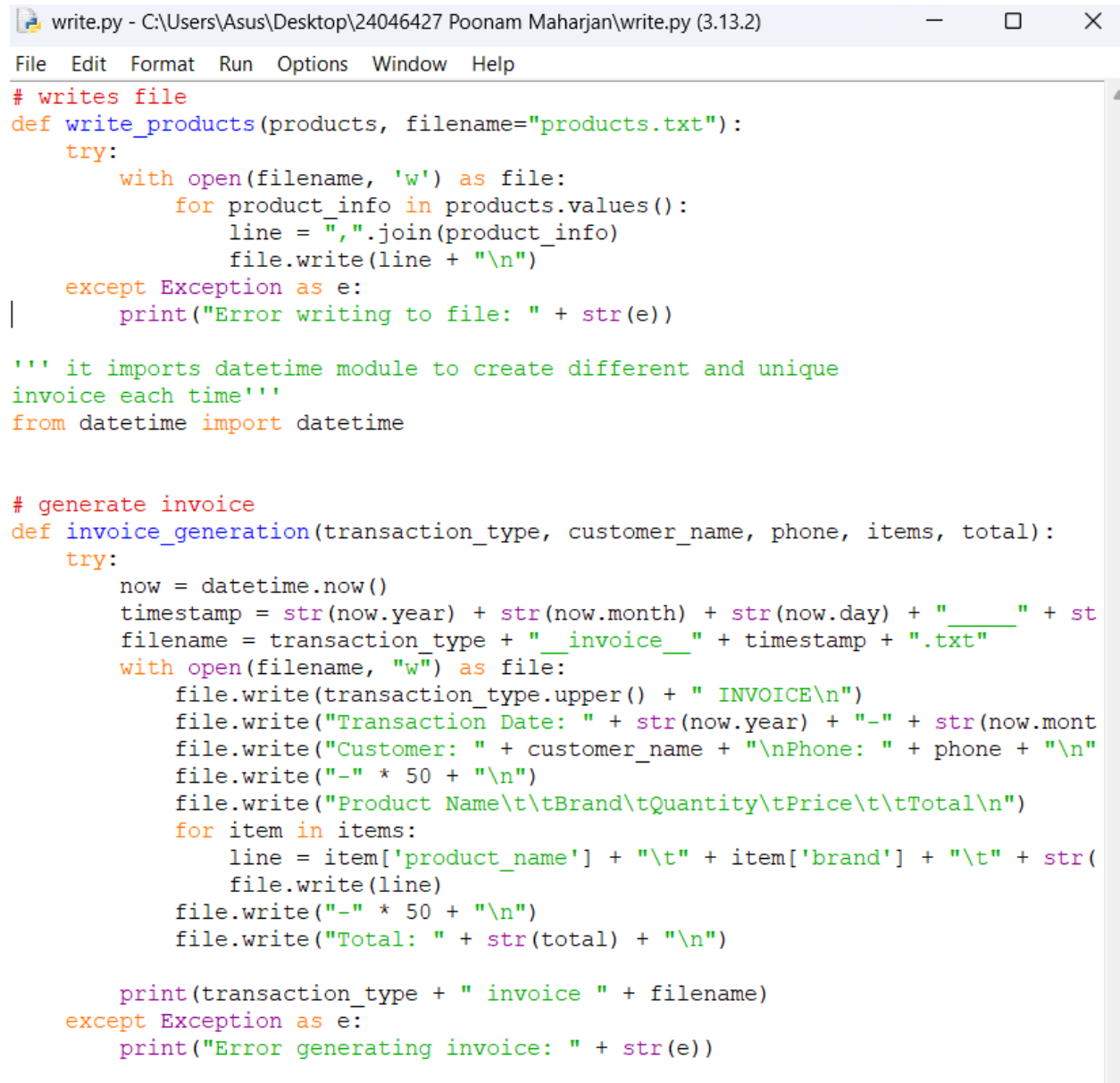


Figure 11 read.py in shell

If the file is not found, it provides a FileNotFoundError stating file not found.

The next is write.py, where there is a function called write_products(products), which saves the updated products and writes dictionary back into products.txt file, this .py

includes another function which is `invoice_generator`, which creates unique .txt files of both purchase and sales using current time and date. This content, both sales and restocking of the products, gives vendor name and phone, gives price, quantity and total.



```

write.py - C:\Users\Asus\Desktop\24046427 Poonam Maharjan\write.py (3.13.2)
File Edit Format Run Options Window Help
# writes file
def write_products(products, filename="products.txt"):
    try:
        with open(filename, 'w') as file:
            for product_info in products.values():
                line = ",".join(product_info)
                file.write(line + "\n")
    except Exception as e:
        print("Error writing to file: " + str(e))

''' it imports datetime module to create different and unique
invoice each time'''
from datetime import datetime

# generate invoice
def invoice_generation(transaction_type, customer_name, phone, items, total):
    try:
        now = datetime.now()
        timestamp = str(now.year) + str(now.month) + str(now.day) + "_" + str(
            now.hour) + str(now.minute) + str(now.second) + ".txt"
        filename = transaction_type + "_invoice_" + timestamp + ".txt"
        with open(filename, "w") as file:
            file.write(transaction_type.upper() + " INVOICE\n")
            file.write("Transaction Date: " + str(now.year) + "-" + str(now.month) + "-" + str(now.day) + "\n")
            file.write("Customer: " + customer_name + "\nPhone: " + phone + "\n")
            file.write("-" * 50 + "\n")
            file.write("Product Name\t\tBrand\tQuantity\tPrice\t\tTotal\n")
            for item in items:
                line = item['product_name'] + "\t" + item['brand'] + "\t" + str(
                    item['quantity']) + "\t" + str(item['price']) + "\t" + str(
                    item['total']) + "\n"
                file.write(line)
            file.write("-" * 50 + "\n")
            file.write("Total: " + str(total) + "\n")

        print(transaction_type + " invoice " + filename)
    except Exception as e:
        print("Error generating invoice: " + str(e))

```

Figure 12 Write.py in shell

The `operation.py` next starts with the `sell_product(product)` function, as it asks the user input for name, phone number, and it displays the available products with selling price = cost * 2 logics, it allows the selection of products and its quantity, and it also applies the logic of Buy 2 get 1 free offer, the stocks reduces as it is purchased accordingly and is added when it is restocked, later it calls the `invoice_generation()` after the sale.

```

''' import invoice_generation and write_products function from write '''
from write import invoice_generation, write_products

# Sell product to the customer
def sell_product(products):
    try:
        while True:
            customer_name = input("Enter the name: ")
            if customer_name.replace(" ", "").isalpha() and len(customer_name) > 2: # customer name should be more than 2 letter
                break
            else:
                print("Enter a valid Name.")

        while True:
            phone_number = input("Enter phone number: ")
            if phone_number.isdigit() and len(phone_number) == 10: #phone number entered by customer should be of 10 digits
                break
            else:
                print("Enter valid number of digits.")

        cart = []
        grand_total = 0

        while True:
            print("\nThe available Products are listed below:\n")
            print("S.N.\tProduct Name\t\tBrand\t\t\tQuantity\t\tPrice")
            print("_____")
            for product_id, product_info in products.items():
                product_name = product_info[0]
                brand = product_info[1]
                stock = product_info[2]
                price = int(product_info[3]) * 2
                print(str(product_id) + "\t" + product_name + "\t\t" + brand + "\t\t" + stock + "\t\t" + str(price))
            print("_____")

            while True:
                selected = input("Enter the Product ID to sell: ")
                if selected.isdigit():
                    selected_id = int(selected)
                    if selected_id in products:
                        break
                    else:
                        print("The ID entered is not available, Please enter a valid product ID.")
                else:
                    print("Invalid ID, The entered ID should be of positive number.")

            try:

```

Figure 13 1st for selling products in operate.py

```

try:
    quantity = int(input("Enter Quantity to Sell: "))
except ValueError:
    print("Invalid quantity. enter the integer.")
    continue

available_stock = int(products[selected_id][2])
free_items = quantity // 3
total_required = quantity + free_items

if total_required > available_stock:
    print("Not sufficient Quantity.") #the product quantity is not sufficient
    continue

products[selected_id][2] = str(available_stock - total_required)
unit_price = int(products[selected_id][3]) * 2
item_total = unit_price * quantity
grand_total += item_total

cart.append({
    "product_name": products[selected_id][0],
    "brand": products[selected_id][1],
    "quantity": quantity,
    "unit_price": unit_price,
    "total": item_total
})

more = input("Want to sell more products (y/n): ").lower()
if not more == 'y': # if the user says no, break the program and create an invoice.
    break

invoice_generation("sale", customer_name, phone_number, cart, grand_total)

except Exception as e:
    print("An error occurred: " + str(e))

```

Figure 14 2nd selling products in opration.py

There is one more function operation.py in restock_product(products), it asks for user to input the name, and phone number, and displays the available products, allows to restock of products, and updates the price and quantity of products and calls invoice_generatrations(), which creates unique invoices

```

# Restocking function
#function
def restock_product(products):
    try:
        while True:
            vendor_name = input("Enter the name of vendor: ")
            if vendor_name.replace(" ", "").isalpha() and len(vendor_name) > 2: #the entered name should be more than 2 letters
                break
            else:
                print("Enter valid name")

        while True:
            phone = input("Enter the phone number: ")
            if phone.isdigit() and len(phone) == 10: # the entered digit should be of 10 digits
                break
            else:
                print("Please enter a valid phone number.")

        cart = []
        total_cost = 0

        while True:
            print("\nWe have the products listed below available\n")
            print("S.N.\tProduct Name\tBrand\tStock\tPrice")
            print("-----")
            for product_id, product_info in products.items():
                print(str(product_id) + "\t" + product_info[0] + "\t" + product_info[1] + "\t" + product_info[2] + "\t" + product_info[3])
            print("-----")

            while True:
                enter_product_id = input("Enter product ID to restock the product: ")
                if enter_product_id.isdigit():
                    selected_id = int(enter_product_id)
                    if selected_id in products:
                        break
                else:
                    print("The entered ID does not exist, please enter a valid ID.")
            else:
                print("Invalid ID, The entered ID should be of positive number.")

            try:
                quantity = int(input("Enter the Quantity to add: "))
            except ValueError:
                print("Invalid quantity. Please enter a number/ digits.")
                continue

```

Figure 15 1st restock the products in shell

```

        price = input("New cost price: ")
        if price:
            products[selected_id][3] = price

        unit_price = int(products[selected_id][3])
        total = unit_price * quantity
        total_cost += total

        products[selected_id][2] = str(int(products[selected_id][2]) + quantity)

        cart.append({
            "product_name": products[selected_id][0],
            "brand": products[selected_id][1],
            "quantity": quantity,
            "unit_price": unit_price,
            "total": total
        })

        more = input("Restock more items? (y/n): ").lower()
        if not more == 'y': #if users says no, the program breaks and creates an invoice
            break

        invoice_generation("restock", vendor_name, phone, cart, total_cost)

    except Exception as e:
        print("Error during restocking: " + str(e))

```

Figure 16 2nd restock the products in shell

The screenshot shows a Python application window titled 'sale_invoice_2025514_85934.txt' and an IDE window titled 'IDLE Shell 3.13.2*'. The application displays a 'SALE INVOICE' for a transaction on 2025-5-14 at 34:59:8. The customer is 'poo' with phone number 4712589632. The invoice lists one item: Vitamin C serum, Brand: Garnier, Quantity: 50, Price: 240, Total: 12000.

The IDE shows the code for the application, which includes a menu with options: 1 --> Sell The Available Products, 2 --> Purchase and Restock Products, 3 --> Exit. The user has chosen option 1, entered the name 'poo' and phone number '4712589632'. The application then displays a list of available products:

S.N.	Product Name	Brand	Quantity	Price
1	Vitamin C serum	Garnier	294	240
2	Skin Cleanser	Cetaphil	100	560
3	Sunscreen	Aqualogica	200	1400
4	Facemask	AquaHoney	400	200
5	Lip balm	Lips	200	400

The user has entered product ID 1 and quantity 50. The application has generated a sale invoice for sale_invoice_2025514_85934.txt.

Figure 17 generation invoice after sale

The screenshot shows a Python application window titled 'restock_invoice_2025514_857.txt' and an IDE window titled 'IDLE Shell 3.13.2*'. The application displays a 'RESTOCK INVOICE' for a transaction on 2025-5-14 at 39:57:8. The customer is 'POO' with phone number 6665554447. The invoice lists one item: Vitamin C serum, Brand: Garnier, Quantity: 50, Price: 130, Total: 6500.

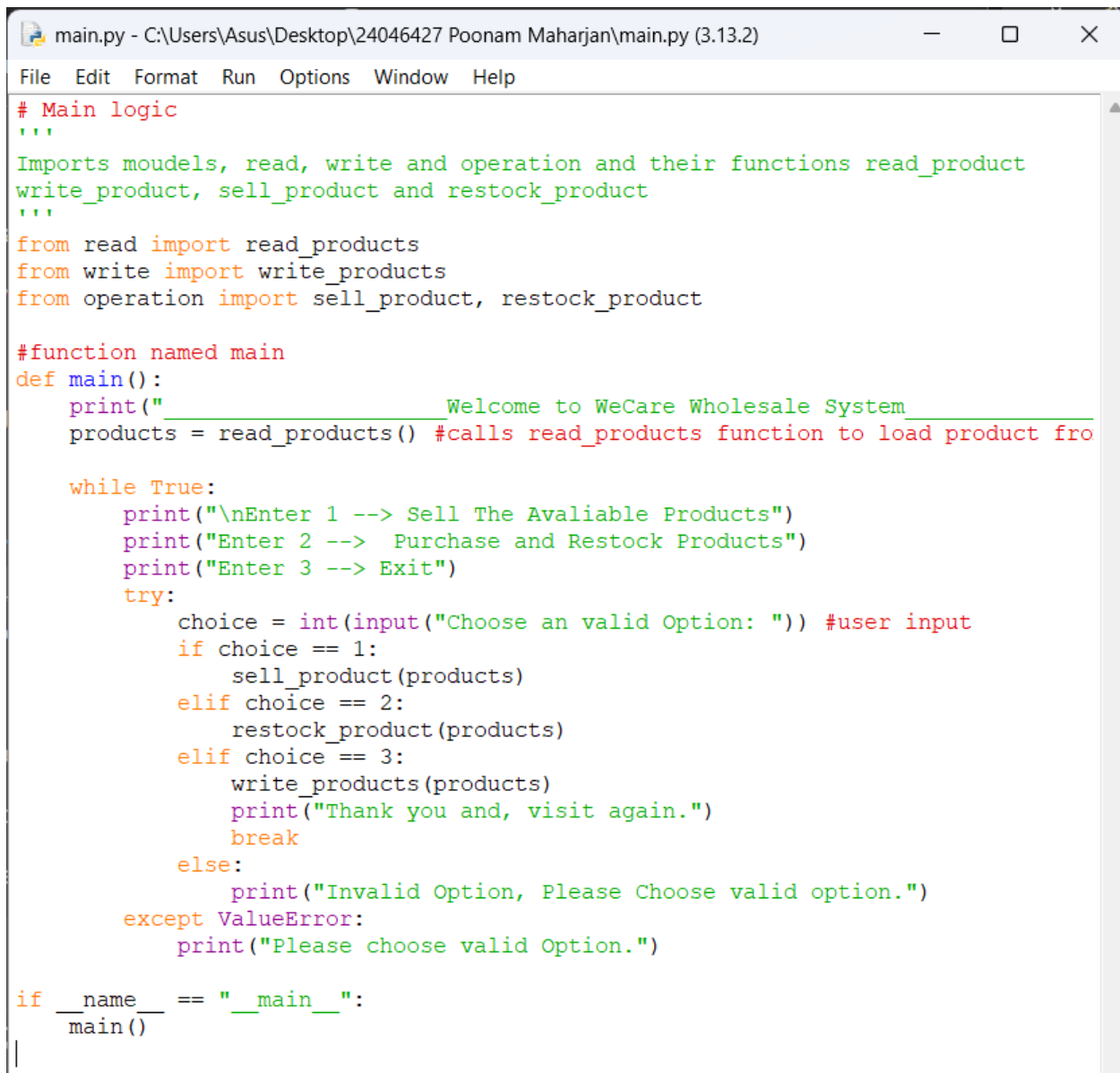
The IDE shows the code for the application, which includes a menu with options: 1 --> Sell The Available Products, 2 --> Purchase and Restock Products, 3 --> Exit. The user has chosen option 2, entered the name 'POO' and phone number '6665554447'. The application then displays a list of products available for restocking:

S.N.	Product Name	Brand	Stock	Price
1	Vitamin C serum	Garnier	294	120
2	Skin Cleanser	Cetaphil	100	280
3	Sunscreen	Aqualogica	200	700
4	Facemask	AquaHoney	400	100
5	Lip balm	Lips	200	200

The user has entered product ID 1 and quantity 50. The application has generated a restock invoice for restock_invoice_2025514_85739.txt.

Figure 18 generating invoice after restocking

The last one to and the most important one is main.py and the function is made called main(), it shows the welcome message, and displays the overall menu, sell, Restock and exit, upon getting the user input on the option, it calls sell_product() or restock_products(), and upon exiting it calls write_products() to save the update and displays the exit message



```
main.py - C:\Users\Asus\Desktop\24046427 Poonam Maharjan\main.py (3.13.2)
File Edit Format Run Options Window Help

# Main logic
'''
Imports moudels, read, write and operation and their functions read_product
write_product, sell_product and restock_product
'''

from read import read_products
from write import write_products
from operation import sell_product, restock_product

#function named main
def main():
    print("_____Welcome to WeCare Wholesale System_____")
    products = read_products() #calls read_products function to load product fro

    while True:
        print("\nEnter 1 --> Sell The Avaliable Products")
        print("Enter 2 --> Purchase and Restock Products")
        print("Enter 3 --> Exit")
        try:
            choice = int(input("Choose an valid Option: ")) #user input
            if choice == 1:
                sell_product(products)
            elif choice == 2:
                restock_product(products)
            elif choice == 3:
                write_products(products)
                print("Thank you and, visit again.")
                break
            else:
                print("Invalid Option, Please Choose valid option.")
        except ValueError:
            print("Please choose valid Option.")

if __name__ == "__main__":
    main()
```

Figure 19 Main.py in shell

```

===== RESTART: C:\Users\Asus\Desktop\24046427 Poonam Maharjan\main.py =====
Welcome to WeCare Wholesale System

Enter 1 --> Sell The Available Products
Enter 2 --> Purchase and Restock Products
Enter 3 --> Exit
Choose an valid Option: 1
Enter the name: poo
Enter phone number: 4712589632

The available Products are listed below:

S.N.      Product Name      Brand      Quantity      Price
-----
1      Vitamin C serum      Garnier      294      240
2      Skin Cleanser      Cetaphil      100      560
3      Sunscreen      Aqualogica      200      1400
4      Facemask      AquaHoney      400      200
5      Lip balm      Lips      200      400

Enter the Product ID to sell: 1
Enter Quantity to Sell: 50
Want to sell more products (y/n): n
sale invoice sale_invoice_2025514_85934.txt

Enter 1 --> Sell The Available Products
Enter 2 --> Purchase and Restock Products
Enter 3 --> Exit
Choose an valid Option: 3
Thank you and, visit again.

```

Figure 20 Exiting the program

4. Testing

4.1 Test 1

Objective	To show the use of try and except for validating phone number
Action	Invalid input was given for the Number (less than 10 digits and a string)
Expected Result	A validation message should be displayed, and the user should be prompted again
Actual Result	A validation message was shown, and user was able to enter the Phone number again
Conclusion	It was successful

Table 5 Test 1

```

Welcome to WeCare Wholesale System
Enter 1 --> Sell The Available Products
Enter 2 --> Purchase and Restock Products
Enter 3 --> Exit
Choose an valid Option: 1
Enter the name: poo
Enter phone number: afje
Enter valid number of digits.
Enter phone number: 985632014
Enter valid number of digits.
Enter phone number: |

```

Figure 21 try and except for validation

4.2 Test 2

Objective	To provide negative and non-existent product id's, during product purchase and sale
Action	Negative value and non-existed values were given as input
Expected Result	A Message should be shown stating that product id does not exist, and positive number is required.
Actual Result	The message was correctly displayed, indicating the ID was invalid and it must be a positive number
Conclusion	It was successful

Table 6 test 2

The available Products are listed below:

S.N.	Product Name	Brand	Quantity	Price
1	Vitamin C serum	Garnier	228	240
2	Skin Cleanser	Cetaphil	100	560
3	Sunscreen	Aqualogica	200	1400
4	Facemask	AquaHoney	400	200
5	Lip balm	Lips	200	400

Enter the Product ID to sell: -1
 Invalid ID, The entered ID should be of positive number.
 Enter the Product ID to sell: |

Figure 22 Negative value for sale

The available Products are listed below:

S.N.	Product Name	Brand	Quantity	Price
1	Vitamin C serum	Garnier	294	240
2	Skin Cleanser	Cetaphil	100	560
3	Sunscreen	Aqualogica	200	1400
4	Facemask	AquaHoney	400	200
5	Lip balm	Lips	200	400

Enter the Product ID to sell: 50
 The ID entered is not available, Please enter a valid product ID.

Figure 23 Invalid ID for sale

We have the products listed below available

S.N.	Product Name	Brand	Stock	Price
1	Vitamin C serum	Garnier	294	120
2	Skin Cleanser	Cetaphil	100	280
3	Sunscreen	Aqualogica	200	700
4	Facemask	AquaHoney	400	100
5	Lip balm	Lips	200	200

Enter product ID to restock the product: -1
 Invalid ID, The entered ID should be of positive number.
 Enter product ID to restock the product: |

Figure 24 Negative value for purchase

We have the products listed below available

S.N.	Product Name	Brand	Stock	Price
1	Vitamin C serum	Garnier	294	120
2	Skin Cleanser	Cetaphil	100	280
3	Sunscreen	Aqualogica	200	700
4	Facemask	AquaHoney	400	100
5	Lip balm	Lips	200	200

Enter product ID to restock the product: -1
 Invalid ID, The entered ID should be of positive number.
 Enter product ID to restock the product: 6
 The entered ID does not exist, please enter a valid ID.
 Enter product ID to restock the product: |

Figure 25 Invalid ID for purchase

4.3 Test 3

Objective	To generate a file after purchasing multiple products, displaying the full process in the shell and in text file
Action	More than one product was purchased, the full purchase process was displayed in shell and saved in a text file
Expected Result	Multiple products should be purchased, and complete process should be displayed in shell, and saved in a text file
Actual Result	Product were purchased, the full process was shown in the shell, and details were stored in a text file.
Conclusion	It was successful

Table 7 test 3

```

Enter 3 --> Exit
Choose an valid Option: 2
Enter the name of vendor: poo
Enter the phone number: 9998884445

We have the products listed below available

S.N.      Product Name      Brand          Stock   Price
-----
1         Vitamin C serum    Garnier        294     120
2         Skin Cleanser         Cetaphil       100     280
3         Sunscreen              Aqualogica    200     700
4         Facemask               AquaHoney     400     100
5         Lip balm                Lips           200     200

Enter product ID to restock the product: 1
Enter the Quantity to add: 50
New cost price: 120
Restock more items? (y/n): y

We have the products listed below available

S.N.      Product Name      Brand          Stock   Price
-----
1         Vitamin C serum    Garnier        344     120
2         Skin Cleanser         Cetaphil       100     280
3         Sunscreen              Aqualogica    200     700
4         Facemask               AquaHoney     400     100
5         Lip balm                Lips           200     200

Enter product ID to restock the product: 2
Enter the Quantity to add: 20
New cost price: 280
Restock more items? (y/n): n
restock invoice restock__invoice__2025514____62830.txt

```

Figure 26 Invoice generation for purchasing in shell

```
restock_invoice_2025514__628
File Edit View

RESTOCK INVOICE
Transaction Date: 2025-5-14__30-28-6
Customer: poo
Phone: 9998884445
-----
Product Name          Brand    Quantity    Price
Vitamin C serum    Garnier      50      120      6000
Skin Cleanser      Cetaphil     20      280      5600
-----
Total: 11600
```

Figure 27 Purchased multiple items saved in .txt file

4.4 Test 4

Objective	To generate a file after selling multiple products, displaying the full process in the shell and in text file
Action	More than one product was sold, the full purchase process was displayed in shell and saved in a text file
Expected Result	Multiple products should be sold, and complete process should be displayed in shell, and saved in a text file
Actual Result	Product were successfully sold, the full process was shown in the shell, and details were stored in a text file.
Conclusion	It was successful

Table 8 test 4

```

Enter 1 --> Sell The Available Products
Enter 2 --> Purchase and Restock Products
Enter 3 --> Exit
Choose an valid Option: 1
Enter the name: poo
Enter phone number: 8885552221

The available Products are listed below:

S.N.      Product Name          Brand                Quantity      Price
-----
1         Vitamin C serum        Garnier              294           240
2         Skin Cleanser          Cetaphil             100           560
3         Sunscreen              Aqualogica          200          1400
4         Facemask                 AquaHoney            400           200
5         Lip balm                  Lips                 200           400

Enter the Product ID to sell: 1
Enter Quantity to Sell: 50
Want to sell more products (y/n): y

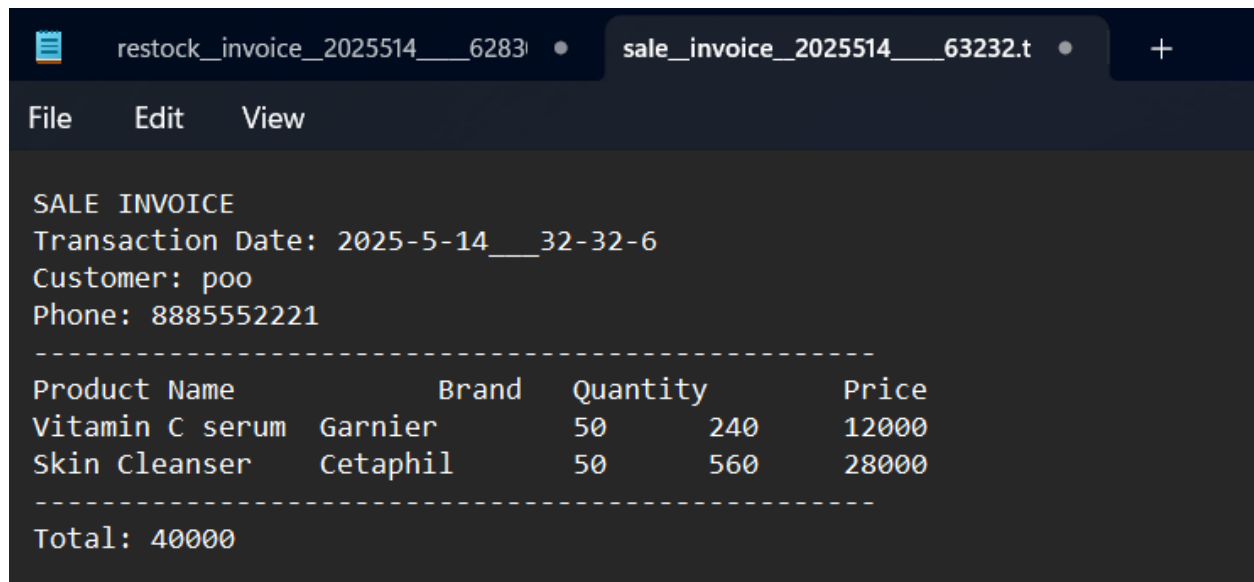
The available Products are listed below:

S.N.      Product Name          Brand                Quantity      Price
-----
1         Vitamin C serum        Garnier              228           240
2         Skin Cleanser          Cetaphil             100           560
3         Sunscreen              Aqualogica          200          1400
4         Facemask                 AquaHoney            400           200
5         Lip balm                  Lips                 200           400

Enter the Product ID to sell: 2
Enter Quantity to Sell: 50
Want to sell more products (y/n): n
sale invoice sale__invoice__2025514____63232.txt

```

Figure 28 Invoice generation for sales

A screenshot of a text editor window with a dark theme. The title bar shows two tabs: 'restock_invoice_2025514____6283' and 'sale_invoice_2025514____63232.t'. The 'View' menu is open. The text content is as follows:

```
SALE INVOICE
Transaction Date: 2025-5-14____32-32-6
Customer: poo
Phone: 8885552221
-----
Product Name      Brand    Quantity  Price
Vitamin C serum  Garnier    50      240    12000
Skin Cleanser    Cetaphil   50      560    28000
-----
Total: 40000
```

Figure 29 Sold products saved in .txt file

4.5 Test 5

Objective	To verify that products stock is updated correctly, added when purchasing and deducted when selling, and both should shown in .txt file and in shell.
Action	Products were purchased and sold, stock was updated accordingly and displayed in the shell and saved in the .txt file
Expected Result	Product stock should increase on purchase and decrease on sale, and both processes should appear in the .txt and shell
Actual Result	Product stock was correctly updated, added on purchase and deducted on sale, and shown in both shell and .txt file
Conclusion	It was successful

Table 9 test 5

```

*IDLE Shell 3.13.2*
File Edit Shell Debug Options Window Help
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\Asus\Desktop\24046427 Poonam Maharjan\main.py =====
Welcome to WeCare Wholesale System

Enter 1 --> Sell The Available Products
Enter 2 --> Purchase and Restock Products
Enter 3 --> Exit
Choose an valid Option: 1
Enter the name: poo
Enter phone number: 9856321471

The available Products are listed below:

S.N.      Product Name      Brand      Quantity      Price
-----
1         Vitamin C serum      Garnier      294          240
2         Skin Cleanser         Cetaphil      100          560
3         Sunscreen             Aqualogica    200          1400
4         Facemask              AquaHoney     400          200
5         Lip balm              Lips          200          400

Enter the Product ID to sell: 1
Enter Quantity to Sell: 50
Want to sell more products (y/n): n
sale invoice sale  invoice  2025514      63715.txt

```

Figure 30 Invoice creation for first sell product

```

Enter 1 --> Sell The Available Products
Enter 2 --> Purchase and Restock Products
Enter 3 --> Exit
Choose an valid Option: 1
Enter the name: poo
Enter phone number: 5469871231

The available Products are listed below:

S.N.      Product Name      Brand      Quantity      Price
-----
1         Vitamin C serum      Garnier      228          240
2         Skin Cleanser         Cetaphil      100          560
3         Sunscreen             Aqualogica    200          1400
4         Facemask              AquaHoney     400          200
5         Lip balm              Lips          200          400

```

Figure 31 Product updated on next sell

```

restock_invoice_2025514____6283l • sale_invoice_2025514____63232.tx • sale_invoice_2025514____63715.t • +
File Edit View

SALE INVOICE
Transaction Date: 2025-5-14__15-37-6
Customer: poo
Phone: 9856321471
-----
Product Name      Brand    Quantity    Price
Vitamin C serum  Garnier    50      240    12000
-----
Total: 12000

```

Figure 32 Products shown in .txt file

```

_____Welcome to WeCare Wholesale System_____

Enter 1 --> Sell The Availiable Products
Enter 2 --> Purchase and Restock Products
Enter 3 --> Exit
Choose an valid Option: 2
Enter the name of vendor: poo
Enter the phone number: 9997775558

We have the products listed below available

S.N.    Product Name    Brand            Stock    Price
-----
1      Vitamin C serum  Garnier          294     120
2      Skin Cleanser    Cetaphil         100     280
3      Sunscreen        Aqualogica       200     700
4      Facemask         AquaHoney        400     100
5      Lip balm         Lips             200     200

Enter product ID to restock the product: 2
Enter the Quantity to add: 50
New cost price: 280
Restock more items? (y/n): y

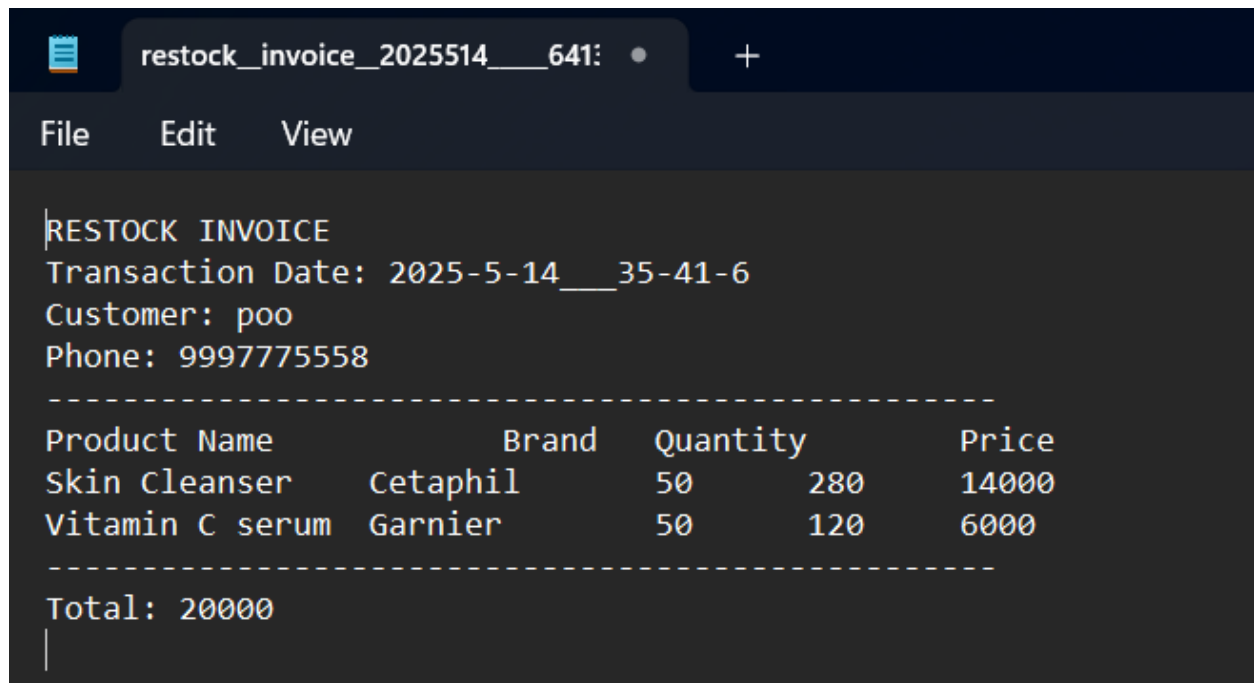
We have the products listed below available

S.N.    Product Name    Brand            Stock    Price
-----
1      Vitamin C serum  Garnier          294     120
2      Skin Cleanser    Cetaphil         150     280
3      Sunscreen        Aqualogica       200     700
4      Facemask         AquaHoney        400     100
5      Lip balm         Lips             200     200

Enter product ID to restock the product:

```

Figure 33 Product added and updated in shell



The screenshot shows a text editor window with a dark theme. The title bar at the top reads "restock_invoice_2025514____641:". Below the title bar is a menu bar with "File", "Edit", and "View". The main text area contains the following content:

```
RESTOCK INVOICE
Transaction Date: 2025-5-14____35-41-6
Customer: poo
Phone: 9997775558
-----
Product Name          Brand    Quantity    Price
Skin Cleanser         Cetaphil    50          280      14000
Vitamin C serum       Garnier     50          120      6000
-----
Total: 20000
|
```

Figure 34 34 Updated shown in .txt file

5. Conclusion

The conceptual understanding, this course has provided is very much needed, as on the completion of this course it teaches and provides overall knowledge on the data structures, algorithms, flowcharts, pseudocodes and more. With the use of most popular programming language Python, it gave quite an experience in it, like skills for coding, making lists, dictionaries, exception handling techniques.

It provided the use of logics to use, about the loops to understand the real-world application. At the end, it taught on how to write clear sets of code and dividing of the code for easier understanding, like we have created different .py files (read.py, write.py, operation.py and main.py), and later imported it. Gave a proper knowledge on handling exceptions, to build user-friendly applications. The unique invoice generator, for each sell and restocking of products, had help on maintaining the logs of the transactions and how important those data are.

In conclusion, With the Real-world scenario, this coursework is both practical and applicable to industry level use. It deals with file structures, logics, exceptions, and problem- solving experiences enhancing our skills, Overall, this assessment helped in analytic thinking, build confident coding and provided a solid foundation for python programming language.

6. Appendix

```
# reads file called product.txt
def read_products(filename='products.txt'):
    products = {}
    try:
        """search for product.txt file and if not
        found throws file not found error """
        with open(filename, "r") as file:
            info = file.readlines()
            product_number = 1 #the ID starts with 1
            for line in info:
                line = line.replace("\n", "")
                product_data = line.split(",")
                products[product_number] = product_data
                product_number += 1
    except FileNotFoundError:
        print("File not found.")
    return products

# writes file
def write_products(products, filename="products.txt"):
    try:
        with open(filename, 'w') as file:
            for product_info in products.values():
                line = ",".join(product_info)
                file.write(line + "\n")
    except Exception as e:
        print("Error writing to file: " + str(e))

""" it imports datetime module to create different and unique
invoice each time"""
from datetime import datetime

# generate invoice
def invoice_generation(transaction_type, customer_name, phone, items, total):
    try:
        now = datetime.now()
        timestamp = str(now.year) + str(now.month) + str(now.day) + "_____" +
str(now.hour) + str(now.minute) + str(now.second) #shows the year, month and day,
similarly the time, hour, minutes and second
        filename = transaction_type + "__invoice__" + timestamp + ".txt"
        with open(filename, "w") as file:
            file.write(transaction_type.upper() + " INVOICE\n")
```

```

        file.write("Transaction Date: " + str(now.year) + "-" + str(now.month) + "-" +
str(now.day) + "___" + str(now.second) + "-" + str(now.minute) + "-" + str(now.hour) +
"\n")
        file.write("Customer: " + customer_name + "\nPhone: " + phone + "\n")
        file.write("-" * 50 + "\n")
        file.write("Product Name\t\tBrand\tQuantity\tPrice\t\tTotal\n")
        for item in items:
            line = item['product_name'] + "\t" + item['brand'] + "\t" + str(item['quantity']) +
"\t" + str(item['unit_price']) + "\t" + str(item['total']) + "\n"
            file.write(line)
        file.write("-" * 50 + "\n")
        file.write("Total: " + str(total) + "\n")

    print(transaction_type + " invoice " + filename)
except Exception as e:
    print("Error generating invoice: " + str(e))

''' import invoice_generation and write_products function from write '''
from write import invoice_generation, write_products

# Sell product to the customer
def sell_product(products):
    try:
        while True:
            customer_name = input("Enter the name: ")
            if customer_name.replace(" ", "").isalpha() and len(customer_name) > 2: #
customer name should be more than 2 letter
                break
            else:
                print("Enter a valid Name.")

        while True:
            phone_number = input("Enter phone number: ")
            if phone_number.isdigit() and len(phone_number) == 10: #phone number
entered by customer should be of 10 digits
                break
            else:
                print("Enter valid number of digits.")

        cart = []
        grand_total = 0

        while True:
            print("\nThe available Products are listed below:\n")
            print("S.N.\tProduct Name\t\tBrand\t\t\tQuantity\tPrice")

```

```
print("_____")
_____
    for product_id, product_info in products.items():
        product_name = product_info[0]
        brand = product_info[1]
        stock = product_info[2]
        price = int(product_info[3]) * 2
        print(str(product_id) + "\t" + product_name + "\t\t" + brand + "\t\t" + stock +
"\t\t" + str(price))
```

```
print("_____")
_____

while True:
    selected = input("Enter the Product ID to sell: ")
    if selected.isdigit():
        selected_id = int(selected)
        if selected_id in products:
            break
        else:
            print("The ID entered is not available, Please enter a valid product ID.")
    else:
        print("Invalid ID, The entered ID should be of positive number.")
```

```
try:
    quantity = int(input("Enter Quantity to Sell: "))
except ValueError:
    print("Invalid quantity. enter the integer.")
    continue
```

```
available_stock = int(products[selected_id][2])
free_items = quantity // 3
total_required = quantity + free_items
```

```
if total_required > available_stock:
    print("Not sufficient Quantity.") #the product quantity is not sufficient
    continue
```

```
products[selected_id][2] = str(available_stock - total_required)
unit_price = int(products[selected_id][3]) * 2
item_total = unit_price * quantity
grand_total += item_total
```

```
cart.append({
    "product_name": products[selected_id][0],
```



```

        "brand": products[selected_id][1],
        "quantity": quantity,
        "unit_price": unit_price,
        "total": item_total
    })

    more = input("Want to sell more products (y/n): ").lower()
    if not more == 'y': # if the user says no, break the program and create an invoice.
        break

    invoice_generation("sale", customer_name, phone_number, cart, grand_total)

except Exception as e:
    print("An error occurred: " + str(e))

# Restocking function
#function
def restock_product(products):
    try:
        while True:
            vendor_name = input("Enter the name of vendor: ")
            if vendor_name.replace(" ", "").isalpha() and len(vendor_name) > 2: #the
entered name should be more than 2 letters
                break
            else:
                print("Enter valid name")

        while True:
            phone = input("Enter the phone number: ")
            if phone.isdigit() and len(phone) == 10: # the entered digit should be of 10 digits
                break
            else:
                print("Please enter a valid phone number.")

        cart = []
        total_cost = 0

        while True:
            print("\nWe have the products listed below available\n")
            print("S.N.\tProduct Name\tBrand\t\tStock\tPrice")

    print("_____")
    _____)
        for product_id, product_info in products.items():

```

```
print(str(product_id) + "\t" + product_info[0] + "\t" + product_info[1] + "\t" +
product_info[2] + "\t" + product_info[3])
```

```
print("_____")
```

```
while True:
```

```
    enter_product_id = input("Enter product ID to restock the product: ")
```

```
    if enter_product_id.isdigit():
```

```
        selected_id = int(enter_product_id)
```

```
        if selected_id in products:
```

```
            break
```

```
        else:
```

```
            print("The entered ID does not exist, please enter a valid ID.")
```

```
    else:
```

```
        print("Invalid ID, The entered ID should be of positive number.")
```

```
try:
```

```
    quantity = int(input("Enter the Quantity to add: "))
```

```
except ValueError:
```

```
    print("Invalid quantity. Please enter a number/ digits.")
```

```
    continue
```

```
price = input("New cost price: ")
```

```
if price:
```

```
    products[selected_id][3] = price
```

```
unit_price = int(products[selected_id][3])
```

```
total = unit_price * quantity
```

```
total_cost += total
```

```
products[selected_id][2] = str(int(products[selected_id][2]) + quantity)
```

```
cart.append({
```

```
    "product_name": products[selected_id][0],
```

```
    "brand": products[selected_id][1],
```

```
    "quantity": quantity,
```

```
    "unit_price": unit_price,
```

```
    "total": total
```

```
})
```

```
more = input("Restock more items? (y/n): ").lower()
```

```
if not more == 'y': #if users says no, the program breaks and creates an invoice
```

```
    break
```

```
invoice_generation("restock", vendor_name, phone, cart, total_cost)
```

```

except Exception as e:
    print("Error during restocking: " + str(e))

# Main logic
'''
Imports moudels, read, write and operation and their functions read_product
write_product, sell_product and restock_product
'''

from read import read_products
from write import write_products
from operation import sell_product, restock_product

#function named main
def main():
    print("_____Welcome to WeCare Wholesale
System_____")
    products = read_products() #calls read_products function to load product from .txt file

    while True:
        print("\nEnter 1 --> Sell The Avaliable Products")
        print("Enter 2 --> Purchase and Restock Products")
        print("Enter 3 --> Exit")
        try:
            choice = int(input("Choose an valid Option: ")) #user input
            if choice == 1:
                sell_product(products)
            elif choice == 2:
                restock_product(products)
            elif choice == 3:
                write_products(products)
                print("Thank you and, visit again.")
                break
            else:
                print("Invalid Option, Please Choose valid option.")
        except ValueError:
            print("Please choose valid Option.")

if __name__ == "__main__":
    main()

```

7. Bibliography

Anon., n.d. [Online].

byjus, 2025. *byjus*. [Online]

Available at: <https://byjus.com/govt-exams/microsoft-word/>

[Accessed 13 may 2025].

Coursera , 2023. *coursera*. [Online]

Available at: [https://www.coursera.org/gb/articles/what-is-python-used-for-a-beginners-guide-to-using-](https://www.coursera.org/gb/articles/what-is-python-used-for-a-beginners-guide-to-using-python?utm_medium=sem&utm_source=gg&utm_campaign=b2c_apac_x_multi_ftcof_career-academy_cx_dr_bau_gg_pmax_pr_s3_en_m_hyb_25-04_x&campaignid=22420706570&adgroupid=&deviceid=22420706570)

[python?utm_medium=sem&utm_source=gg&utm_campaign=b2c_apac_x_multi_ftcof_career-academy_cx_dr_bau_gg_pmax_pr_s3_en_m_hyb_25-04_x&campaignid=22420706570&adgroupid=&deviceid=22420706570](https://www.coursera.org/gb/articles/what-is-python-used-for-a-beginners-guide-to-using-python?utm_medium=sem&utm_source=gg&utm_campaign=b2c_apac_x_multi_ftcof_career-academy_cx_dr_bau_gg_pmax_pr_s3_en_m_hyb_25-04_x&campaignid=22420706570&adgroupid=&deviceid=22420706570)

[Accessed 13 may 2025].

Dancuk, M., 2023. *phoenixNAP*. [Online]

Available at: [https://phoenixnap.com/kb/boolean-data-](https://phoenixnap.com/kb/boolean-data-type#:~:text=A%20boolean%20is%20a%20data,algebra%2C%20named%20after%20George%20Bool)

[type#:~:text=A%20boolean%20is%20a%20data,algebra%2C%20named%20after%20George%20Bool](https://phoenixnap.com/kb/boolean-data-type#:~:text=A%20boolean%20is%20a%20data,algebra%2C%20named%20after%20George%20Bool)

[Accessed 13 May 2025].

Dekera, k., 2025. *researchgate*. [Online]

Available at: https://www.researchgate.net/figure/Standard-Flowchart-Symbols_fig1_338671462

[Accessed 13 may 2025].

docs.python.org, 2025. *docs.python.org*. [Online]

Available at: <https://docs.python.org/3/library/idle.html>

[Accessed 11 may 2025].

ElHousieny, R., 2023. *ranyel.medium*. [Online]

Available at: <https://ranyel.medium.com/introduction-to-strings-in-python-18caf6e02152>

[Accessed 11 may 2025].

ElHousieny, R., 2023. *ranyel.medium*. [Online]

Available at: <https://ranyel.medium.com/introduction-to-strings-in-python-18caf6e02152>

[Accessed 13 May 2025].

geeksforgeeks, 2024. *geeksforgeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/what-is-a-notepad/>

[Accessed 11 may 2025].

geeksforgeeks, 2024. *geeksforgeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/primitive-and-non-primitive-data-types-in-javascript/>

[Accessed 13 may 2025].

geeksforgeeks, 2025. *geeksforgeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/cpp-data-types/>

[Accessed 13 May 2025].

geeksforgeeks, 2025. *geeksforgeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/difference-between-float-and-double/>

[Accessed 13 may 2025].

geeksforgeeks, 2025. *geeksforgeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/python-lists/>

[Accessed 13 May 2025].

geeksforgeeks, 2025. *geeksforgeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/python-dictionary/>
[Accessed 13 May 2025].

geeksforgeeks, 2025. *geeksforgeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/sets-in-python/>
[Accessed 13 May 2025].

geeksforgeeks, 2025. *geeksforgeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/tuples-in-python/>
[Accessed 13 May 2025].

geeksforgeeks, n.d. [Online].

Gillis, A. S., 2025. *techtarget*. [Online]
Available at: <https://www.techtarget.com/whatis/definition/algorithm>
[Accessed 13 may 2025].

lenovo, 2025. *lenovo*. [Online]
Available at: <https://www.lenovo.com/in/en/glossary/what-is-notepad/?orgRef=https%253A%252F%252Fwww.google.com%252F>
[Accessed 13 may 2025].

Lucidchart, 2025. *lucidchart*. [Online]
Available at: <https://www.lucidchart.com/pages/what-is-a-flowchart-tutorial>

Paraschiv, L., 2023. *fotc*. [Online]
Available at: <https://fotc.com/blog/draw-io-online-guide/#:~:text=Draw.io%20is%20a%20free,Google%20Workspace%20or%20Gmail%20account>
[Accessed 13 may 2025].

python, 2025. *python.org*. [Online]
Available at: <https://www.python.org/doc/essays/blurb/>

unstop, 2025. *unstop*. [Online]
Available at: <https://unstop.com/blog/python-idle#:~:text=Python%20IDLE%20is%20a%20development,in%20using%20this%20beautiful%20tool>
[Accessed 13 may 2025].

w3schools, 2025. *w3schools*. [Online]
Available at: https://www.w3schools.com/python/python_numbers.asp
[Accessed 13 May 2025].

w3schools, 2025. *w3schools*. [Online]
Available at: https://www.w3schools.com/python/python_booleans.asp
[Accessed 13 May 2025].

Wambua, k., 2023. *medium*. [Online]
Available at: https://www.researchgate.net/figure/Standard-Flowchart-Symbols_fig1_338671462
[Accessed 13 may 2025].

Wambua, k., n.d. [Online].